numcompute-stream: A Streaming Machine Learning Framework

NumCompute Assignment 2.2 — University of Adelaide

Omkar Thombre Student ID: 1982851

1 Design Decisions

`numcompute-stream` is a streaming machine learning framework built from scratch using only NumPy, matplotlib, and the Python standard library. The system implements the full pipeline of incremental learning: streaming preprocessing, a Hoeffding Tree classifier, an Online Random Forest ensemble, and Adaptive Random Forest (ARF) drift detection. This section describes the key algorithmic and engineering decisions that shaped the implementation.

1.1 Hoeffding Tree (VFDT)

`HoeffdingTreeClassifier` implements the Very Fast Decision Tree algorithm of Domingos & Hulten (2000). The central insight of VFDT is that a streaming learner never needs to store raw samples: each leaf maintains *sufficient statistics* that can be updated in $O(1)$ per sample and from which the best split can be evaluated at any time.

Each leaf stores a class-count vector and a `RunningStats` object for every (feature, class) pair. `RunningStats` uses Welford's one-pass algorithm to accumulate the mean and sum of squared deviations without materialising the sample array. When split candidates are evaluated, per-class Gaussian parameters are read from these accumulators and fed into the vectorised `_info_gains_for_feature` function, which computes information gain across all candidate thresholds in a single NumPy pass.

Split commitment is governed by the Hoeffding bound: after observing n samples at a leaf, a split is committed when the gap between the best and second-best impurity gain exceeds $\varepsilon = \sqrt{R^2 \ln(1/\delta)/2n}$. The range parameter R depends on the criterion: for Gini, $R = 1$; for entropy, $R = \log_2(K)$ where K is the number of classes. We use $\delta = 10^{-7}$ as the default split confidence. A tie-breaking threshold τ (default 0.05) allows early splitting when the bound is already tight and the two top candidates are nearly indistinguishable, preventing a leaf from hoarding samples when additional data are unlikely to change the outcome.

1.2 Online Random Forest

`OnlineRandomForestClassifier` builds an ensemble of `HoeffdingTreeClassifier` instances following the online bagging scheme of Oza & Russell (2001). For each incoming sample, each tree receives a Poisson($\lambda = 1$) weight drawn independently. Samples with weight zero are used as out-of-bag (OOB) observations for that tree, tracking its accuracy without any hold-out set. Since $E[\text{Poisson}(1)] = 1$, each sample contributes on average exactly once per tree — matching the expectation of batch bootstrap — but the scheme is fully online: no sample buffer is required.

Diversity comes from two sources: each tree is assigned a random feature subset of size `max_features` at construction time, and the independent Poisson weights ensure that trees see different replications of each chunk. Prediction aggregates trees by hard-vote (default) or probability-averaged soft-vote.

1.3 ADWIN and Adaptive Random Forest

The ARF variant of `OnlineRandomForestClassifier` adds concept-drift detection following Gomes et al. (2017). Each tree maintains two independent `ADWIN` detectors, both fed the binary OOB correctness signal (1 for correct, 0 for incorrect).

The *warning* detector uses `warning_delta=0.01`, a looser threshold that fires on early signs of degradation. When it fires, a background candidate tree is instantiated and fits in parallel on subsequent samples without yet contributing to the ensemble vote. The *drift* detector uses `drift_delta=0.001`, a stricter threshold requiring stronger evidence. When it fires, the original tree is replaced: if a candidate exists, it becomes the new tree; otherwise a fresh tree is created. This two-threshold state machine prevents resets triggered by transient noise. Both detectors enforce a minimum window of 100 OOB samples before evaluating any cut point, preventing spurious detection on the leading edge of a stream.

`ADWIN` itself (Bifet & Gavaldà, 2007) maintains a sliding window of bounded size (`max_window_size=5000`), tests all candidate cut points for a significant shift in the window mean, and drops the older sub-window if drift is confirmed. Our implementation uses a plain deque rather than exponential histograms, giving $O(W^2)$ worst-case update cost (Section 4.3).

1.4 NumPy-only Compliance

The assignment prohibits pandas and scipy. Two places required non-trivial workarounds. CSV ingestion uses the standard `csv` module rather than `pandas.read_csv`: `load_csv` reads each row as strings, converts columns to `float64` with explicit `np.nan` propagation, and concatenates into a NumPy array. The normal CDF required for Gaussian split evaluation wraps `math.erf` via `np.frompyfunc` to create a ufunc that accepts NumPy arrays:

```
_verf = np.frompyfunc(_math.erf, 1, 1)
def _normal_cdf(z):
```

```
return 0.5 * (1.0 + _verf(z / math.sqrt(2.0))).astype(np.float64)
```

This replaces `scipy.special.erf` while still vectorising over threshold arrays. Majority-class prediction uses `np.argmax` on the class-count vector rather than `scipy.stats.mode`.

1.5 Uniform Streaming Interface

All components share a protocol enforced by abstract base classes in `base.py`: `BaseTransformer` requires `partial_fit`, `transform`, `reset`; `BaseEstimator` requires `partial_fit`, `predict`, `reset`; `BaseMetric` requires `update`, `result`, `reset`; `BaseDriftDetector` requires `update` and `reset`. This uniformity makes `Pipeline` and `StreamTrainer` trivial: both work with any combination of components without modification.

2 Testing and Edge Cases

2.1 Per-module Test Coverage

The full test suite consists of 267 tests across 10 test files. Coverage was measured with `pytest --cov=numcompute_stream`.

Module	Stmts	Miss	Coverage	Tests
`base.py`	35	2	94%	—
`drift.py`	66	5	92%	12
`ensemble.py`	210	16	92%	33
`io.py`	83	12	86%	20
`metrics.py`	201	12	94%	41
`pipeline.py`	64	8	88%	15
`preprocessing.py`	204	25	88%	28
`stats.py`	163	16	90%	27
`stream.py`	66	1	98%	—
`tree.py`	301	21	93%	51
`utils.py`	38	0	100%	23
`visualise.py`	82	0	100%	17
Total	**1,523**	**118**	**92%**	**267**

Uncovered lines are predominantly error branches in `io.py` and argument-validation paths in `preprocessing.py` reachable only through specific file-encoding edge cases; the core algorithmic paths are fully exercised.

2.2 Categories of Edge Cases

Numerical stability. `RunningStats` is tested against `np.var` on normally distributed, constant, and single-sample sequences. A constant stream must produce variance exactly zero; a single sample must raise on unbiased variance. `StreamingHistogram` quantile estimates are tested against `np.quantile` on 10,000-sample random data within a tolerance of 0.05.

Missing data. `Imputer` is tested with columns that are entirely `np.nan` on the first chunk, which must not raise and must produce a valid imputed output once sufficient data arrive. The Hoeffding Tree skips NaN values when updating per-feature statistics but still counts the sample in the class distribution; NaN-containing rows at predict time are routed left by convention.

Shape mismatches. All transformers and estimators raise `ValueError` on the first call if the feature count does not match what was fitted. 1-D inputs auto-reshape to 2-D in `predict`.

Streaming semantics. `StandardScaler` is tested to confirm that statistics accumulate correctly across multiple `partial_fit` calls. `Accuracy` and `RollingMetric` are tested to confirm `reset()` returns to initial state. `StreamTrainer`'s prequential evaluation skips prediction on the first chunk (no fitted model yet).

Drift detection. `ADWIN` is tested with a hand-crafted step-function stream where drift must be detected, and with a stationary stream where it must not fire. `OnlineRandomForestClassifier` is tested to confirm that drift events increment `drift_event_count_` and that tree replacement produces a fresh tree.

2.3 Mathematical Correctness Tests

Four tests verify the underlying mathematics directly. `test_var_matches_numpy` asserts agreement between `RunningStats` and `np.var` to 10 decimal places, guarding against off-by-one errors in the Welford M2 accumulator. `test_hoeffding_bound_gini` and `test_hoeffding_bound_entropy` assert the closed-form bound for known (n, δ) pairs — this test caught a real bug during development where R was incorrectly set to 1 for both criteria, causing entropy-based trees to split too aggressively. `test_binary_tree_node_identity_after_split` asserts the structural invariant `n_nodes == 2·n_leaves - 1` after any sequence of splits.

2.4 Test-Driven Development

Two bugs were caught purely by tests before reaching any end-to-end run. The first was the Hoeffding bound R-parameter bug noted above; the unit test for the entropy case failed immediately, pointing to the missing substitution. Without this test the bug would have been invisible at the integration level — the tree would still produce predictions, just with splits committed earlier than the statistical guarantee warranted. The second was a Welford reset bug

where _n was reset but _mean and _M2 were not. A test that fits, resets, then re-fits caught this immediately. Since RunningStats is used inside every leaf of every tree in the forest, this bug would have corrupted per-leaf statistics silently.

3 Benchmark Results

3.1 Methodology

All benchmarks use time.perf_counter and report the minimum wall-clock time over five runs on identical pre-generated input. The minimum-of-five convention excludes OS scheduling jitter and cache-cold first runs. Loop baselines are written in idiomatic Python with no deliberate pessimisation; vectorised implementations are the production code paths from numcompute_stream. Hardware: Apple Silicon (darwin 25.5.0), Python 3.14.3, NumPy 2.x.

3.2 RunningStats Welford Update

Setup: 100,000-sample one-dimensional chunk fed into a fresh RunningStats accumulator.

Implementation	Time (ms)	Speedup
Loop (scalar Welford)	15.96	1.00×
Vectorised (Chan parallel merge)	0.062	**258.98**×

The vectorised path uses Chan's parallel merge: the batch mean is computed as X.mean(axis=0) and the batch M2 as ((X - batch_mean) ** 2).sum(axis=0), then merged into the running accumulator. Both operations are BLAS-routed calls on contiguous float64 memory and exploit SIMD instructions. The scalar loop pays Python interpreter dispatch for every one of 100,000 samples and cannot exploit SIMD or cache prefetching across elements.

3.3 StandardScaler Transform

Setup: 100,000 × 8 matrix with pre-fitted mean and std; transform only.

Implementation	Time (ms)	Speedup
Loop (row-by-row)	44.84	1.00×
Vectorised (broadcast)	0.833	**53.80**×

The vectorised implementation computes $(X - \mu)/\sigma$ as a single broadcasting operation. NumPy lays out the matrix in row-major order, so the subtraction and division traverse memory linearly, keeping 800,000 float64 values in cache through one linear scan. The row loop forces 100,000 Python frame invocations and breaks spatial locality.

3.4 Info-Gain Across Candidate Thresholds

Setup: 30 candidate thresholds, 3 classes, 1,000 samples per leaf.

Implementation	Time (ms)	Speedup
Loop (per-threshold)	0.1436	1.00×
Vectorised (_info_gains_for_feature)	0.0262	**5.49**×

_info_gains_for_feature stacks all threshold-conditional class counts into an (n_thresh, n_classes) matrix, computes the Gaussian CDF and child impurities in one pass, and masks degenerate splits without Python branching. The loop version calls math.erf and _gini_scalar separately for each of the 30 thresholds.

3.5 Summary

The 5.49× speedup is substantially lower than the other two benchmarks because math.erf is itself a transcendental function computed by a minimax polynomial approximation. When the per-element operation is algorithmically expensive, Python dispatch overhead becomes a smaller fraction of total time. The pattern is consistent across all three benchmarks: vectorisation matters most when the inner per-element operation is simple arithmetic. For numcompute-stream, the highest-frequency code paths — per-sample statistics updates, per-chunk scaling — benefit disproportionately, while lower-frequency paths such as split evaluation yield smaller but still meaningful improvements.

4 Reflections

4.1 What Went Well

Committing to a true VFDT from the outset — rather than a mini-batch approximation that buffers samples — was the right call. The sufficient-statistics design (Welford accumulators per leaf, no stored samples) kept memory bounded by construction and forced clean separation between statistics accumulation and split evaluation. This made the tree code easier to test in isolation: a leaf's statistics are a deterministic function of the samples seen, independent of when or whether a split occurs.

The uniform `partial_fit` / `transform` / `predict` / `update` / `reset` interface, enforced by abstract base classes, paid dividends throughout the project. `Pipeline` and `StreamTrainer` were written once and work with any combination of components. Adding `OnlineRandomForestClassifier` did not require changes to the evaluation harness; it dropped in as a `BaseEstimator` like any other.

Test-driven development caught at least two bugs that would have been silent at the integration level: the Hoeffding bound R-parameter error and the Welford reset bug. Both were caught within minutes of writing the relevant production code, demonstrating the value of writing unit tests alongside the function rather than after the full system was assembled.

4.2 What Was Harder Than Expected

ADWIN's $O(W^2)$ cut search was more of a practical constraint than initially appreciated. The canonical implementation (Bifet & Gavaldà, 2007) uses exponential histograms that reduce update cost to $O(\log W)$. Implementing these in pure Python is non-trivial, and we chose instead to cap `max_window_size` at 5,000. On streams with slow drift the window fills quickly and every update triggers a full scan; this is acceptable for the demo-scale streams in this project but would be a hard limit at production scale.

The ARF state machine was more complex than the description in Gomes et al. (2017) suggests. Three state variables interact per tree — warning detector, drift detector, candidate tree — and the transitions between *stable*, *warning*, and *drift* states must handle edge cases: what happens if the drift detector fires before a candidate was created? Getting these transitions correct required careful test iteration.

An unexpected empirical finding was that the Online Random Forest with `max_features='sqrt'` underperformed a single Hoeffding Tree on the 8-feature Electricity dataset. With `sqrt` each tree sees only 2–3 features, which is insufficient diversity for such a low-dimensional dataset. Setting `max_features=6` restored the expected ensemble advantage. This is a well-known sensitivity of Random Forest to feature-subset size, but it was a concrete reminder that the `sqrt` default is calibrated for higher-dimensional data.

4.3 Limitations

$O(W^2)$ ADWIN. The plain-deque window implementation has $O(W^2)$ update cost in the worst case. The canonical $O(\log W)$ solution requires exponential histograms, which were out of scope. The cap at `max_window_size=5000` is pragmatic, not principled.

Binary-only AUC. `StreamingAUC` computes the area under the ROC curve via trapezoidal approximation on a running score histogram, but only supports binary classification. Multiclass AUC would require a one-vs-rest decomposition or a direct multiclass formulation.

Axis-aligned splits only. `HoeffdingTreeClassifier` considers only univariate threshold splits. Oblique splits would improve accuracy on rotated boundaries but complicate the splitting routine.

No model serialisation. All model state lives in Python objects in memory. There is no mechanism to save a fitted model to disk and resume in a later process — a significant limitation for any deployment scenario.

4.4 What I Would Do Differently

Building a parameterised synthetic drift generator earlier would have accelerated development. Adjustable drift timing, drift type (abrupt vs. gradual), and number of classes would have made ARF unit tests more rigorous and the demo notebook more informative. Instead, drift scenarios were constructed ad hoc in test fixtures, making them harder to reuse.

Investing in visualisation tooling earlier would have made ADWIN tuning more transparent. The current `visualise.py` was written near the end of the project; a live metric-over-time plot earlier would have surfaced the Online RF / single-tree gap immediately and saved time diagnosing the `max_features` issue.

4.5 Closing

The project delivered a working streaming machine learning library in 3,566 lines of source across 267 tests and 31 commits, achieving 92% line coverage with no external dependencies beyond NumPy and matplotlib. The core algorithms — VFDT, online bagging, and ADWIN-based adaptive drift detection — behave as the cited papers describe, and the vectorised implementations produce meaningful performance improvements over their loop equivalents. The limitations are real and honestly bounded; the framework is not production-ready, but it is a faithful implementation of the theoretical foundations it was designed to demonstrate.

References. Bifet, A., & Gavaldà, R. (2007). Learning from time-changing data with adaptive windowing. *SDM*, 443–448. Domingos, P., & Hulten, G. (2000). Mining high-speed data streams. *KDD*, 71–80. Gomes, H. M., et al. (2017). Adaptive random forests for evolving data stream classification. *Machine Learning*, 106(9–10), 1469–1495. Oza, N. C., & Russell, S. (2001). Online bagging and boosting. *AISTATS*, 105–112.